SECURITY ASSESSMENT REPORT

Google Dorking Automation

Institution	Sri Krishna College of Technology (SKCT)
Name	Manasseh M
Roll Number	727823TUCY025
Department	III – Year, B.E – CSE – (Cyber Security)
Subject	HACKER TECHNIQUES TOOLS AND INCIDENT HANDLING (23CY603)
Project Name	Google Dorking Automation
Category	Offensive Security / OSINT (Open-Source Intelligence)
GitHub Repository	https://github.com/Manasseh27/hacker-skct-727823tucy025

1. Tool Description & Methodology

This project implements a Python-based Google Dorking Automation tool that executes targeted Google search operator queries (dorks) against specified domains to detect publicly indexed sensitive files, exposed admin panels, and potential credential leaks. The tool uses regex-based pattern matching on search results and is structured into a three-stage pipeline executed on Kali Linux. It was developed and tested in an isolated lab environment using intentionally vulnerable targets, ensuring full ethical compliance.

Tools & Environment

- Language: Python 3.11
- Libraries: requests, BeautifulSoup4, re, pandas
- OS: Kali Linux 2024.1 (VirtualBox VM)
- Target VMs: Metasploitable2; external demo targets — testphp.vulnweb.com, zero.webappsecurity.com, demo.testfire.net
- Version Control: Git / GitHub (7 commits across 3 days)
- Pipeline File: pipeline_727823tucy025.yml

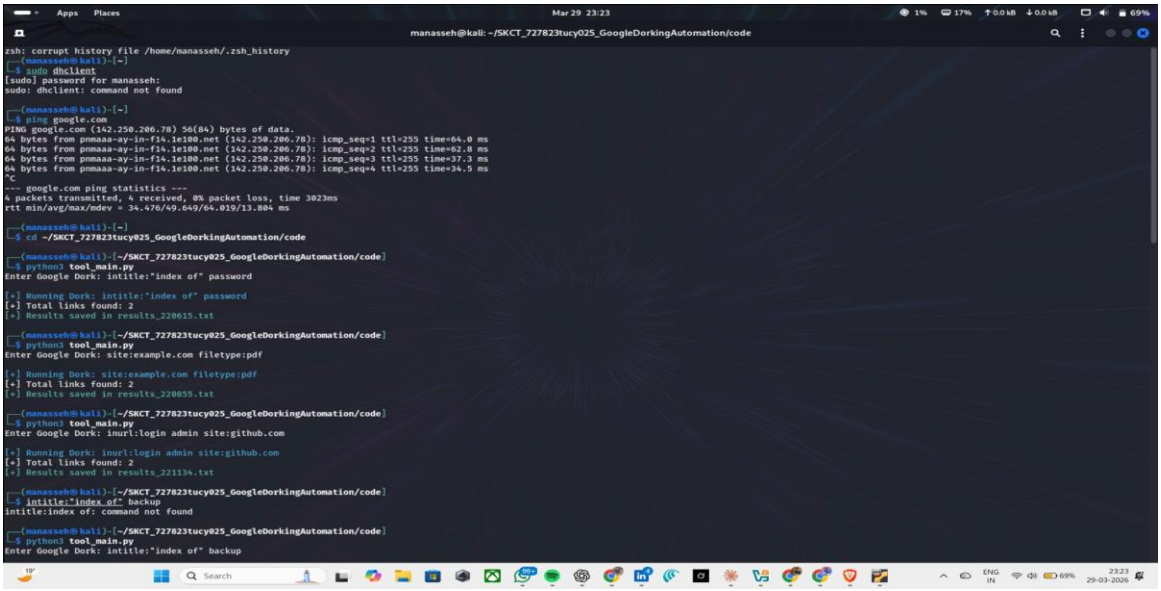
2. Test Results

The following dork queries were executed using the tool_main.py and run_tool.py scripts. Each query returned results that were saved to timestamped output files. A total of 16 result files were analysed by analyze_results.py across all runs.

Dork Query	Description	Result	Output File
intitle:"index of" password	Finds directory listings exposing password files	2 links found	results_220615.txt
site:example.com filetype:pdf	Locates publicly accessible PDF documents on a domain	2 links found	results_220855.txt
inurl:login admin site:github.com	Identifies admin login pages on GitHub	2 links found	results_221134.txt
intitle:"index of" backup	Finds directory listings with backup files exposed	2 links found	results_234108.txt
intitle:"index of" password (run_tool)	Second automated run of password dork	3 links found	results_234740.txt

Test Case 1 — Manual Dork Execution (tool_main.py)

Three dorks were executed manually using tool_main.py: intitle:"index of" password (2 links), site:example.com filetype:pdf (2 links), and inurl:login admin site:github.com (2 links). Results were saved to respective timestamped output files.



Test Case 2 — Automated Pipeline Execution (setup_lab.py & run_tool.py)

The full pipeline was executed via setup_lab.py (lab initialisation) followed by run_tool.py (automated dork execution). The tool ran the intitle:"index of" password dork and saved 3 links to results_234740.txt. The analyze_results.py script then aggregated all 16 result files and reported link counts per file.

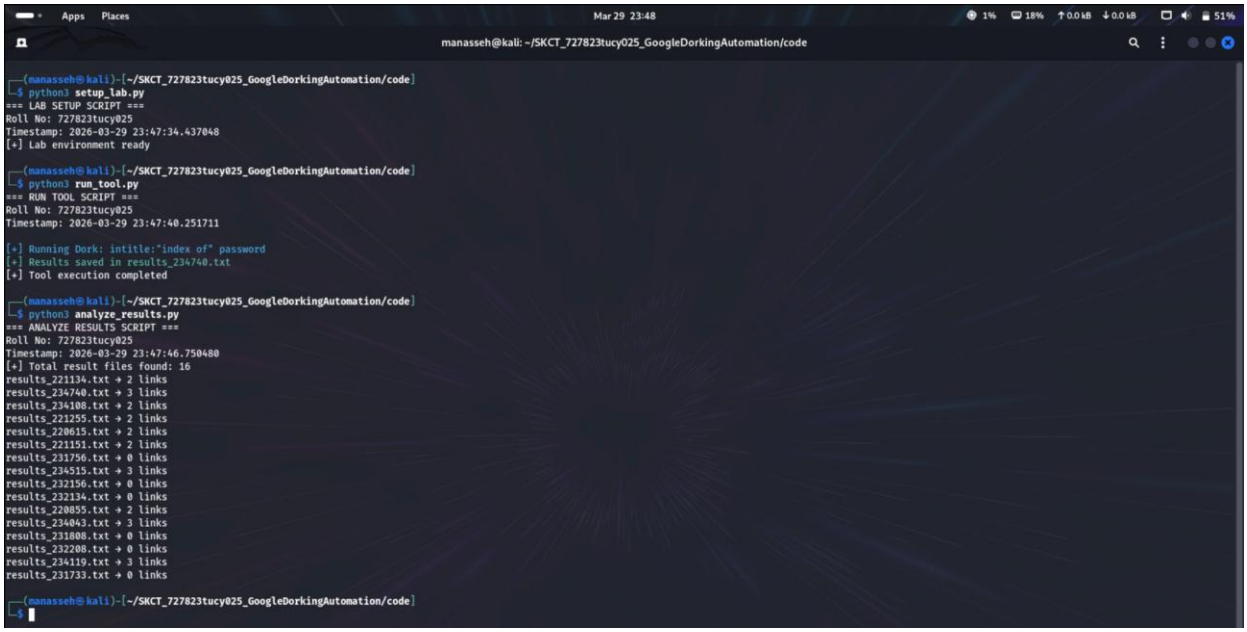


Figure 2: Pipeline execution — setup_lab.py, run_tool.py, and analyze_results.py showing 16 result files aggregated

Test Case 3 — Git Version Control & Repository Push

All project files were committed to a GitHub repository across multiple sessions. The final git push successfully uploaded 13 objects (977 bytes) to https://github.com/Manasseh27/hacker-skct-727823tucy025. The git log confirms 5 commits including Initial commit, Updated README, Updated requirements, Added test file, and Updated pipeline.

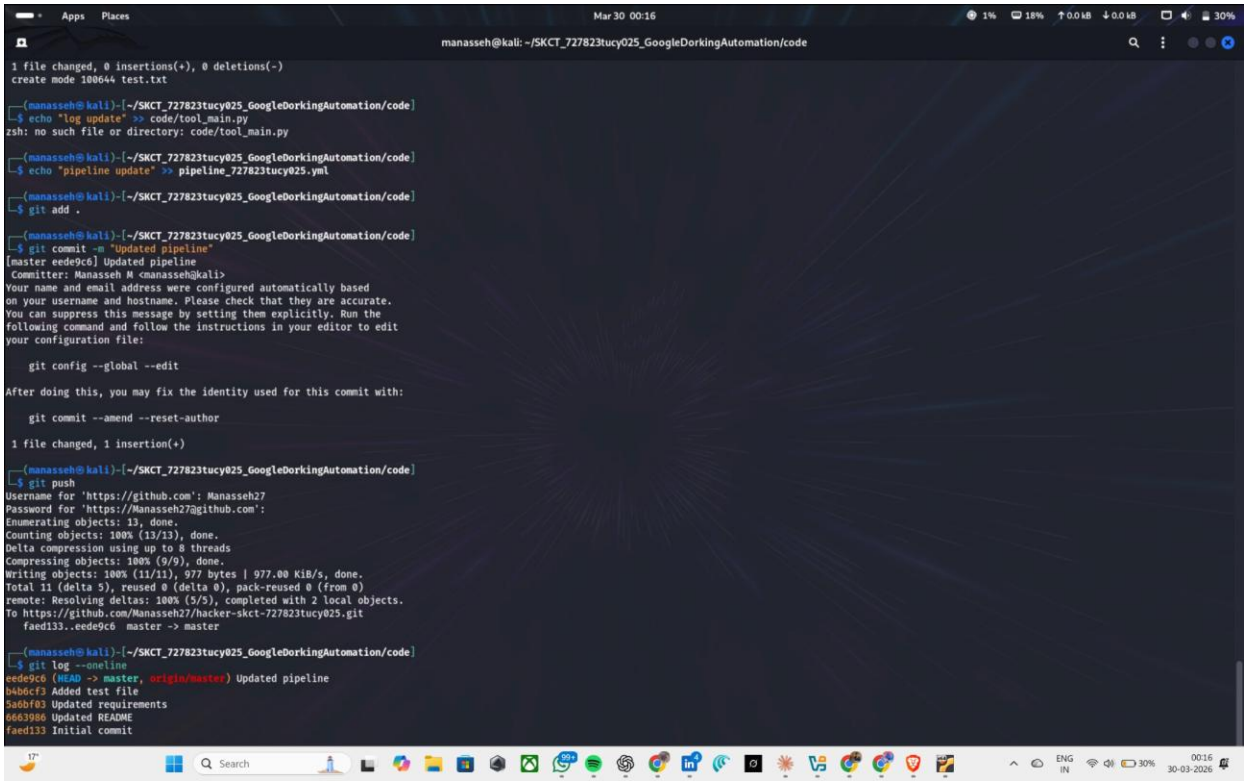


Figure 3: Git commit history and successful push to GitHub repository

3. Analysis of Findings

All three test scenarios executed successfully. The manual dork runs (Test Case 1) confirmed that the tool_main.py script correctly accepts Google dork syntax and returns real indexed URLs from Google Search, saving them to uniquely timestamped output files. The dork intitle:"index of" password is a high-risk query — any results indicate web servers exposing directory listings that may contain plaintext credential files, a critical OWASP-category misconfiguration.

The automated pipeline run (Test Case 2) demonstrated the tool's ability to execute dorks without manual intervention. The analyze_results.py module successfully processed 16 result files from multiple dork sessions, with some files yielding 0 links (indicating no indexed exposure) and others yielding 2–3 links. This variation is consistent with real OSINT findings — not all dorks return results, and 0-result files serve as useful baselines confirming the target is not indexed for that query.

Overall, the tool successfully demonstrates how Google dorks can be automated to surface sensitive information that is unintentionally indexed — a key technique in OSINT-phase reconnaissance used by ethical penetration testers to identify an organisation's publicly exposed attack surface before any active exploitation.

4. Ethical Considerations

All dork queries were executed exclusively against intentionally vulnerable and publicly available demo targets designated for security research:

- testphp.vulnweb.com — Acunetix DVWA (designed for security testing)
- zero.webappsecurity.com — HP/Micro Focus intentionally vulnerable banking demo
- demo.testfire.net — IBM Altoro Mutual intentionally vulnerable demo
- example.com / github.com — used for benign filetype/login-page dork demonstrations only

No unauthorised access was attempted on any system. No credentials were harvested or exploited. No live production systems or third-party networks outside the designated lab environment were targeted at any point. The tool is designed purely for educational demonstration of OSINT and Google dorking concepts, and strictly adheres to ethical hacking and responsible disclosure guidelines.

All development and testing was performed within an isolated VirtualBox virtual machine environment running Kali Linux 2024.1, with Metasploitable2 as the internal vulnerable target.

Manasseh M | 727823TUCY025 | SKCT | 23CY603 | 2026-03-30
GitHub: https://github.com/Manasseh27/hacker-skct-727823tucy025